

Deployment and Operations Handover

This guide is the short operational index. The complete procedure is `docs/runbooks/production-deployment.md`; `deploy.sh` is the executable implementation and must be read before changing deployment behavior.

Production topology

Production is defined by `docker-compose.prod.yml`:

- `backend`: compiled Node/TypeScript application; code is baked into the image. Mounts uploads and logs volumes.
- `frontend`: Vite build served by Nginx; code is baked into the image.
- `postgres`: PostgreSQL 15 with persistent `postgres_data`.
- `redis`: Redis 7 with persistent `redis_data`.
- `clamav`: malware scanner used by attachment scanning.
- `nginx`: public reverse proxy on ports 80/443; mounts Nginx configuration and certificate volumes.
- `certbot`: certificate renewal loop.

The production host/repository/domain details are in the deployment runbook. Access is controlled through the organization's approved SSH and secret-management process; do not place connection secrets in documentation.

Release gates

Production changes require explicit approval. Before any production action:

- 1 Confirm the intended commit and branch.
- 2 Inspect `git diff <previous-production-commit>..HEAD -- backend/prisma/` for schema changes.
- 3 Run backend build, frontend build, frontend typecheck, relevant tests, and `git diff --check`.
- 4 Back up the production database to the repository `backups/` directory and verify the dump exists and is non-empty.
- 5 Decide whether migration/status-audit work is required.
- 6 Ensure the release owner and rollback commit are recorded.

Do not commit or push as part of this handover unless separately requested and approved.

Standard deployment

Run from the repository root after approval:

```
./deploy.sh --no-seed --no-cache --verbose
```

Use `--no-migrate` only when the release has no schema changes and migration status is still checked. Use `--seed` only when approved seed/reference-data changes are part of the release. Production seed behavior must preserve admin-managed configuration (`RETAIN_ADMIN_CONFIG=true`). Never run `reset`, `clear`, `demo-data`, or `force-reset` commands against production.

The script's intended order is:

- 1 Local backend TypeScript pre-flight.
- 2 Pull the approved `dev2.0` revision on production.

- 3 Build and start the backend image.
- 4 Run migration/status-catalog checks.
- 5 Optionally run the guarded seed.
- 6 Build the frontend image.
- 7 Recreate services.
- 8 Verify backend and frontend health.

Build backend and frontend separately because the production host has limited memory. If a build appears to reuse stale layers, use `--no-cache` and verify a unique changed string in `/app/dist/` or the frontend asset bundle.

Nginx routes `/api/*` to the backend, gives the notification stream a long-lived unbuffered connection, routes `/health` to the backend, serves other application paths from the frontend, and redirects HTTP to HTTPS. Backend health endpoints are `/health/live`, `/health/ready`, and `/health`; readiness checks database connectivity and can check Redis. These are defined in `backend/src/app.ts`.

Migration safety

For changes involving `tenant_id`, constraints, existing rows, or legacy workflow/status data, follow the pre-deploy read-only audit and post-deploy verification procedure in the full runbook. If migration deployment fails, stop and inspect the exact migration/data error. Do not blindly run `prisma db push --accept-data-loss`; it is a recovery operation requiring an approved data plan and any required NULL backfill.

The correct manual ordering is: build backend → restart backend with the new image → migration/status audit → approved seed → build frontend → recreate all services.

Post-deployment verification

```
# On production, from the application directory
docker compose -f docker-compose.prod.yml ps

docker exec citadel-cwc-portal-nginx-1 \
  curl -s -o /dev/null -w '%{http_code}' http://backend:3000/health

# From a trusted client
curl -sk https://cwc.citadelgroup.com.my/ -o /dev/null -w '%{http_code}\n'
```

Expected results: backend HTTP 200; public frontend HTTP 200 or 301; containers running/healthy. For schema releases, also run `npx prisma migrate status` inside the backend container and confirm the schema is up to date. Then perform a role-appropriate browser smoke test of the changed feature. A health response proves process availability, not feature correctness.

Diagnostics

Canonical recent production logs are Docker logs, not the potentially stale `backend/logs/` volume:

```
docker compose -f docker-compose.prod.yml logs backend --tail=200 --no-log-prefix
docker compose -f docker-compose.prod.yml logs frontend --tail=50 --no-log-prefix
docker inspect citadel-cwc-portal-backend-1 \
  --format 'RestartCount: {{.RestartCount}} StartedAt: {{.State.StartedAt}} ExitCode: {{.State.ExitCode}}'
```

Check container uptime, restart count, memory, disk, Redis/PostgreSQL health, and the public health endpoint before changing application code. Backend port 3000 is internal; check it through the Nginx container or public proxy.

Common incidents

- **Changes not visible:** verify the image contains the changed code, force-recreate if the container uses an old image, then hard-refresh/try private browsing because `index.html` or browser caches may be stale.
- **Backend repeatedly restarts:** inspect backend logs and restart metadata; common causes include missing required environment variables, migration failure, dependency health, or PDF/Chromium startup issues.
- **Migration failure:** stop, preserve the backup, inspect migration status and SQL/schema names, and use the documented recovery procedure.
- **Duplicate scheduled work:** confirm Redis availability and singleton scheduler configuration before running multiple backend instances.
- **Attachment stuck:** inspect ClamAV health, scanner queue/worker logs, Redis, object storage, and attachment status transitions.
- **PDF export timeout:** inspect BullMQ worker logs and verify the user-scoped Redis result key and Puppeteer runtime inside the backend container.

Rollback

- 1 Identify the last known-good commit from the production repository log.
- 2 Record the incident and decide whether the issue is code-only or also data/schema.
- 3 Check out the approved known-good commit and rebuild backend/frontend separately with `--no-cache`.
- 4 Recreate services and run health/browser/schema verification.
- 5 Restore the verified database backup only when the release changed data/schema and restoration is approved. Never use `prisma migrate reset` as a routine rollback.

Document the final deployed commit, verification results, incident/change reference, and any follow-up migration repair.

Backup and restore references

Use these repository documents and scripts for the full backup/restore procedure and retention expectations:

- `docs/backup-restore-policy.md`
- `docs/prod-db-backup-restore-plan.md`
- `scripts/backup-db.sh`
- `scripts/verify-backup.sh`

Verify the backup before production migrations or data changes. Perform restore exercises in non-production and never overwrite production without explicit approval.